Data Analytics + AI - Assignment 1

Healthcare Data Analysis with Python

Student ID: DA21106
Name: Silvana Carolina Paredes Vanegas

Dataset 1: Patient Records

patient_id	age	province	diagnosis	treatment_duration	readmission
1001	45	Nova Scotia	Diabetes	30	No
1002	62	New Brunswick	Hypertension	45	Yes
1003	38	Nova Scotia	Diabetes	28	No
1004	55	Prince Edward Island	Asthma	NULL	No
1005	71	Newfoundland	Hypertension	60	Yes
1006	49	Nova Scotia	Diabetes	35	No
1007	33	New Brunswick	Asthma	NULL	Yes
1008	58	Nova Scotia	Hypertension	52	Yes

Dataset 2: Treatment Outcomes

patient_id	treatment_type	initial_blood_pressure	final_blood_pressure	patient_satisfaction
1001	Medication	145	128	8.5
1002	Lifestyle Change	160	142	7.2
1003	Medication	152	135	9.0
1004	Therapy	NULL	NULL	8.3
1005	Medication	168	149	6.8
1006	Lifestyle Change	148	138	7.9
1007	Therapy	NULL	NULL	9.2
1008	Medication	155	133	8.7

Setup: Libraries and Datasets

```python
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# ----------------------------------------------------------------
# Datasets:
# ----------------------------------------------------------------

patients_df = pd.DataFrame({
    'patient_id': [1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008],
    'age': [45, 62, 38, 55, 71, 49, 33, 58],
    'province': ['Nova Scotia', 'New Brunswick', 'Nova Scotia', 'Prince Edward Island',
    'Newfoundland', 'Nova Scotia', 'New Brunswick', 'Nova Scotia'],
    'diagnosis': ['Diabetes', 'Hypertension', 'Diabetes', 'Asthma', 'Hypertension',
    'Diabetes', 'Asthma', 'Hypertension'],
    'treatment_duration': [30, 45, 28, np.nan, 60, 35, np.nan, 52],
    'readmission': ['No', 'Yes', 'No', 'No', 'Yes', 'No', 'Yes', 'Yes']
})

outcomes_df = pd.DataFrame({
    'patient_id': [1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008],
    'treatment_type': ['Medication', 'Lifestyle Change', 'Medication', 'Therapy',
    'Medication', 'Lifestyle Change', 'Therapy', 'Medication'],
    'initial_blood_pressure': [145, 160, 152, np.nan, 168, 148, np.nan, 155],
    'final_blood_pressure': [128, 142, 135, np.nan, 149, 138, np.nan, 133],
    'patient_satisfaction': [8.5, 7.2, 9.0, 8.3, 6.8, 7.9, 9.2, 8.7]
})

print(patients_df)
print(outcomes_df)
```

Part 1: NumPy Array Operations (30 points)

1. Create a NumPy array from the 'age' column in the Patient Records dataset. Calculate and print:
A. The mean age
B. The median age
C. The standard deviation of ages

Code:

```
# ----------------------------------------------------------------
# Part 1: NumPy Array Operations
# ----------------------------------------------------------------
# 1. Create a NumPy array from the 'age' column in the Patient Records dataset.
# ----------------------------------------------------------------
age_np = patients_df['age'].to_numpy()
print("\nPart 1: NumPy Array Operations")

# A. Mean age
mean_age = np.mean(age_np)
print("Mean age:", mean_age)

# B. Median age
median_age = np.median(age_np)
print("Median age:", median_age)

# C. Standard deviation of ages
std_age = np.std(age_np)
print("Standard deviation:", std_age)
```

Output:

```
Part 1: NumPy Array Operations
Mean age: 51.375
Median age: 52.0
Standard deviation: 11.82092953197844
```

Explanations:

This code calculates the mean, median, and standard deviation of the patients' ages using NumPy.

2. Create a 2D NumPy array containing patient_id, age, and treatment_duration (replace NULL with 0). Reshape this array to have 3 columns.

Code:

```
# ----------------------------------------------------------------
# 2. Create a 2D NumPy array containing patient_id, age, and
# treatment_duration (replace NULL with 0).
# Reshape this array to have 3 columns.
# ----------------------------------------------------------------

# Select the columns: patient_id, age, and treatment_duration
patients_2d_df = patients_df[['patient_id', 'age', 'treatment_duration']]

# Replace NULL with 0 in treatment_duration column
patients_2d_df = patients_2d_df.fillna(0)

# Convert to 2D NumPy array
patients_2d_np = patients_2d_df.to_numpy()

# Reshape the array to have 3 columns, where -1 means a flexible
# number of rows, and 3 means the number of columns
patients_2d_np = patients_2d_np.reshape(-1, 3)

# Convert back to DataFrame for better display
patients_2d_view_df = pd.DataFrame(
    patients_2d_np, columns=patients_2d_df.columns
)

print("\n2. Create a 2D NumPy array")
print(patients_2d_view_df)
```

Output:

```
2. Create a 2D NumPy array
   patient_id   age  treatment_duration
0      1001.0  45.0                30.0
1      1002.0  62.0                45.0
2      1003.0  38.0                28.0
3      1004.0  55.0                 0.0
4      1005.0  71.0                60.0
5      1006.0  49.0                35.0
6      1007.0  33.0                 0.0
7      1008.0  58.0                52.0
```

Explanations:

This code creates a 2D NumPy array with patient_id, age, and treatment_duration, replacing missing values in treatment_duration with 0.

Then, it reshapes the 2D NumPy array to have three columns. Since the array already has three columns, the shape remains unchanged.

3. Use NumPy to create a boolean mask identifying patients over 60 years old. Print the patient IDs that match this condition.

Code:

```
# ----------------------------------------------------------------
# 3. Use NumPy to create a boolean mask identifying patients over 60 years old.
# Print the patient IDs that match this condition.
# ----------------------------------------------------------------
over_60_mask_np = age_np > 60

print("\n3. Identifying patients over 60 years old")
print(patients_df.loc[over_60_mask_np, ['patient_id', 'age']])
```

Output:

```
3. Identifying patients over 60 years old
   patient_id  age
1        1002   62
4        1005   71
```

Explanations:

This code identifies patients over 60 by creating a boolean mask and using it to filter the records required.

4. Calculate the correlation coefficient between age and treatment_duration using NumPy (excluding NULL values).

Code:

```
# ----------------------------------------------------------------
# 4. Calculate the correlation coefficient between age and
# treatment_duration using NumPy (excluding NULL values).
# ----------------------------------------------------------------
treatment_np = patients_df['treatment_duration'].to_numpy()

# Boolean mask: True where treatment_duration is not NaN
valid_treatment_mask = ~np.isnan(treatment_np)

# Apply mask to both arrays
age_clean_np = age_np[valid_treatment_mask]
treatment_clean_np = treatment_np[valid_treatment_mask]

# Calculate correlation coefficient
correlation_matrix = np.corrcoef(age_clean_np, treatment_clean_np)
correlation_coefficient = correlation_matrix[0, 1]

print("\n4. Correlation coefficient between age and treatment_duration\n")
print(correlation_coefficient)
```

Output:

```
4. Correlation coefficient between age and treatment_duration

0.9508283471535305
```

Explanations:

This code calculates the Pearson correlation coefficient between age and treatment duration, showing the strength and direction of the linear relationship between these two numerical variables.

Values close to 1 indicate a strong positive linear relationship, values close to -1 indicate a strong negative linear relationship, and values near 0 indicate little or no linear relationship.

In this case, <u>the correlation is 0,95</u>, which indicates a strong positive linear relationship between age and treatment duration. This means that, in general, older patients tend to have longer treatment duration.

Part 2: Pandas Data Manipulation (40 points)

5. Load both datasets into Pandas DataFrames. Display the first 5 rows and basic information (data types, non-null counts) for each DataFrame.

Code:

```
# ----------------------------------------------------------------
#Part 2: Pandas Data Manipulation (40 points)
# ----------------------------------------------------------------
#5. Load both datasets into Pandas DataFrames.
# ----------------------------------------------------------------
# Display the first 5 rows and basic information
#(data types, non-null counts) for each DataFrame.
print("\nPart 2: Pandas Data Manipulation")

# Display the first 5 rows
print("\n5.Display the first 5 rows")
print("\nPatients:\n", patients_df.head())
print("\nTreatments:\n", outcomes_df.head())

# Display Dataframes info
print("\nDisplay Dataframe info")
print("\nPatients dataset info:")
patients_df.info()

print("\nTreatments dataset info:")
outcomes_df.info()
```

Output:

Display the first 5 rows:

```
5.Display the first 5 rows

Patients:
    patient_id  age  ... treatment_duration readmission
0        1001   45  ...               30.0          No
1        1002   62  ...               45.0         Yes
2        1003   38  ...               28.0          No
3        1004   55  ...                NaN          No
4        1005   71  ...               60.0         Yes

[5 rows x 6 columns]

Treatments:
    patient_id    treatment_type  ...  final_blood_pressure  patient_satisfaction
0        1001        Medication  ...                 128.0                   8.5
1        1002  Lifestyle Change  ...                 142.0                   7.2
2        1003        Medication  ...                 135.0                   9.0
3        1004           Therapy  ...                   NaN                   8.3
4        1005        Medication  ...                 149.0                   6.8

[5 rows x 5 columns]
```

Display DataFrame Info:

```
Display Dataframe info

Patients dataset info:
<class 'pandas.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   patient_id          8 non-null      int64
 1   age                 8 non-null      int64
 2   province            8 non-null      str
 3   diagnosis           8 non-null      str
 4   treatment_duration  6 non-null      float64
 5   readmission         8 non-null      str
dtypes: float64(1), int64(2), str(3)
memory usage: 516.0 bytes

Treatments dataset info:
<class 'pandas.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   patient_id              8 non-null      int64
 1   treatment_type          8 non-null      str
 2   initial_blood_pressure  6 non-null      float64
 3   final_blood_pressure    6 non-null      float64
 4   patient_satisfaction    8 non-null      float64
dtypes: float64(3), int64(1), str(1)
memory usage: 452.0 bytes
```

Explanations:

The first part shows the first five rows of each dataset to provide an overview of the data and its structure.

The second part displays basic information about each dataset, including column names, data types, and counts of non-null values.

6. Handle missing values:
A. Identify how many NULL values exist in each DataFrame

Code:

```
# ----------------------------------------------------------------
#6. Handle missing values:
# ----------------------------------------------------------------
#A. Identify how many NULL values exist in each DataFrame
# Count nulls
patients_null_counts = patients_df.isnull().sum()
outcomes_null_counts = outcomes_df.isnull().sum()

print("\n6A. Missing values")
print(f"\nPatients Dataframe:\n{patients_null_counts}")
print(f"\nTreatments Dataframe:\n{outcomes_null_counts}")
```

Output:

```
6A. Missing values

Patients Dataframe:
patient_id            0
age                   0
province              0
diagnosis             0
treatment_duration    2
readmission           0
dtype: int64

Treatments Dataframe:
patient_id                0
treatment_type            0
initial_blood_pressure    2
final_blood_pressure      2
patient_satisfaction      0
dtype: int64
```

Explanations:

This code counts the number of missing (NULL/NaN) values in each column of both datasets.
There are NULL values in:
- Patients: treatment_duration (2)
- Treatment: initial_blood_pressure (2) and final_blood_pressure (2)

B. Fill NULL values in 'treatment_duration' with the median duration for that diagnosis

Code:

```
#B. Fill NULL values in 'treatment_duration' with the median duration for that diagnosis
median_duration_dict = {
    'Diabetes': patients_df.loc[patients_df['diagnosis'] == 'Diabetes', 'treatment_duration'].median(),
    'Hypertension': patients_df.loc[patients_df['diagnosis'] == 'Hypertension', 'treatment_duration'].median(),
    'Asthma': patients_df.loc[patients_df['diagnosis'] == 'Asthma', 'treatment_duration'].median()
}

# Apply medians only where treatment_duration is NaN
for diagnosis, median_value in median_duration_dict.items():
    patients_df.loc[
        (patients_df['diagnosis'] == diagnosis) & (
         patients_df['treatment_duration'].isna()),
         'treatment_duration'
    ] = median_value

print(f"\n6B. Fill NULL values with the median duration:\n{patients_df}")
```

Output:

```
6B. Fill NULL values with the median duration:
   patient_id  age  ... treatment_duration readmission
0        1001   45  ...               30.0          No
1        1002   62  ...               45.0         Yes
2        1003   38  ...               28.0          No
3        1004   55  ...                NaN          No
4        1005   71  ...               60.0         Yes
5        1006   49  ...               35.0          No
6        1007   33  ...                NaN         Yes
7        1008   58  ...               52.0         Yes

[8 rows x 6 columns]
```

Explanations:

The code applies the median to each row where the treatment_duration is missing and the diagnosis matches, using a loop over the dictionary of medians.

The median is calculated separately for Diabetes, Hypertension, and Asthma. However, since both Asthma entries have missing treatment_duration, the median for Asthma remains NaN. This will cause problems in the regression model if not handled.

C. Drop rows with NULL blood pressure values in the Treatment Outcomes dataset

Code:

```
#C. Drop rows with NULL blood pressure values in the Treatment Outcomes dataset
outcomes_df = outcomes_df.dropna(subset=['initial_blood_pressure', 'final_blood_pressure'])
print(f"\n6C. Drop rows with NULL blood pressure\n {outcomes_df}")
```

Output:

```
6C. Drop rows with NULL blood pressure
    patient_id    treatment_type  ...  final_blood_pressure  patient_satisfaction
0         1001        Medication  ...                 128.0                   8.5
1         1002  Lifestyle Change  ...                 142.0                   7.2
2         1003        Medication  ...                 135.0                   9.0
4         1005        Medication  ...                 149.0                   6.8
5         1006  Lifestyle Change  ...                 138.0                   7.9
7         1008        Medication  ...                 133.0                   8.7

[6 rows x 5 columns]
```

Explanations:

This code removes rows from the Treatment Outcomes dataset where initial_blood_pressure or final_blood_pressure is missing: two rows meet this condition and are therefore dropped.

7. Merge the two datasets on patient_id using an inner join. Display the resulting DataFrame.

Code:

```
# ----------------------------------------------------------------
#7. Merge the two datasets on patient_id using an inner join.
# Display the resulting DataFrame.
# ----------------------------------------------------------------
merged_df = pd.merge(patients_df, outcomes_df, on='patient_id', how='inner')

#To display all the columns on the screen
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

print(f"\n7.Merge\n {merged_df}")
```

Output:

```
7.Merge
   patient_id  age       province     diagnosis  treatment_duration readmission
0        1001   45    Nova Scotia      Diabetes                30.0          No
1        1002   62  New Brunswick  Hypertension                45.0         Yes
2        1003   38    Nova Scotia      Diabetes                28.0          No
3        1005   71   Newfoundland  Hypertension                60.0         Yes
4        1006   49    Nova Scotia      Diabetes                35.0          No
5        1008   58    Nova Scotia  Hypertension                52.0         Yes

   treatment_type  initial_blood_pressure  final_blood_pressure  patient_satisfaction
      Medication                   145.0                 128.0                   8.5
Lifestyle Change                   160.0                 142.0                   7.2
      Medication                   152.0                 135.0                   9.0
      Medication                   168.0                 149.0                   6.8
Lifestyle Change                   148.0                 138.0                   7.9
      Medication                   155.0                 133.0                   8.7
```

Explanations:

This code performs an inner join of the patients and outcomes datasets on patient_id, creating a combined dataset with all relevant patient and treatment information.

Display settings in Pandas are adjusted to show all columns and expand the output width in the console. This allows the full merged dataset to be viewed without truncation.

Part 3: Introduction to scikit-learn (30 points)

8. Prepare the data for modeling:
A. Create a feature matrix X with columns: age, treatment_duration, initial_blood_pressure, patient_satisfaction

Code:

```
# ----------------------------------------------------------------
# Part 3: Introduction to scikit-learn (30 points)
# ----------------------------------------------------------------
# 8. Prepare the data for modeling:
# ----------------------------------------------------------------
# A. Create a feature matrix X with columns: age, treatment_duration,
# initial_blood_pressure, patient_satisfaction
# ----------------------------------------------------------------
# Remove rows where 'treatment_duration' is NULL before modeling
model_df = merged_df.dropna(
    subset=['treatment_duration']
)

# Create a feature matrix X
X = model_df[
    ['age', 'treatment_duration', 'initial_blood_pressure', 'patient_satisfaction']
]

print("\nPart 3: Introduction to scikit-learn")
print(f"\nFeature matrix X\n{X}")
```

Output:

```
Part 3: Introduction to scikit-learn

Feature matrix X
   age  treatment_duration  initial_blood_pressure  patient_satisfaction
0   45                30.0                   145.0                   8.5
1   62                45.0                   160.0                   7.2
2   38                28.0                   152.0                   9.0
3   71                60.0                   168.0                   6.8
4   49                35.0                   148.0                   7.9
5   58                52.0                   155.0                   8.7
```

Explanations:

This code creates a feature matrix X using the columns age, treatment_duration, initial_blood_pressure, and patient_satisfaction.

***Note**: Rows where treatment_duration is missing are removed to prevent errors in the regression model. In Exercise 6B, the median for Asthma could not be calculated because all its treatment_duration values were missing. Keeping those missing values would cause the model to fail.*

B. Create a target vector y with the readmission status (convert 'Yes'/'No' to 1/0)

Code:

```
# ----------------------------------------------------------------
# B. Create a target vector y with the readmission status
# (convert 'Yes'/'No' to 1/0)
# ----------------------------------------------------------------
y = model_df['readmission'].map({'Yes': 1, 'No': 0})

print(f"\nTarget vector y\n{y}")
```

Output:

```
Target vector y
0    0
1    1
2    0
3    1
4    0
5    1
Name: readmission, dtype: int64
```

Explanations:

This code converts the readmission column into a binary target vector y, mapping Yes to 1 and No to 0. This transformation is necessary because logistic regression requires numerical values for the target variable.

9. Split the data into training (80%) and testing (20%) sets using train_test_split with random_state=42.

Code:

```
# ----------------------------------------------------------------
# 9. Split the data into training (80%) and testing (20%) sets using
# train_test_split with random_state=42.
# ----------------------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining set shape:\n", X_train.shape)
print("\nTest set shape:\n", X_test.shape)
```

Output:

```
Training set shape:
 (4, 4)

Test set shape:
 (2, 4)
```

Explanations:

This code splits the dataset into training (80%) and testing (20%).

The print is showing the shapes of the training and test feature matrices after splitting the data:

- *The training set has 4 rows (samples) and 4 columns (features).*
- *The test set has 2 rows (samples) and 4 columns (features).*

10. Apply StandardScaler from scikit-learn to normalize the features. Print the mean and standard deviation of each feature after scaling.

Code:

```
# ----------------------------------------------------------------
# 10. Apply StandardScaler from scikit-learn to normalize the features.
# Print the mean and standard deviation of each feature after scaling.
# ----------------------------------------------------------------
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nPart 3: Introduction to scikit-learn")
print("\nMean after scaling:", X_train_scaled.mean(axis=0))
print("\nStandard deviation after scaling:", X_train_scaled.std(axis=0))
```

Output:

```
Part 3: Introduction to scikit-learn

Mean after scaling: [0.00000000e+00 0.00000000e+00 5.55111512e-17 2.22044605e-16]

Standard deviation after scaling: [1. 1. 1. 1.]
```

Explanations:

This code applies StandardScaler to normalize the feature values, ensuring that each feature has mean 0 and standard deviation 1. Normalization improves the performance and convergence of the logistic regression model.

11. Use scikit-learn's LogisticRegression to train a simple model predicting readmission. Report the accuracy score on the test set.

Code:

```
# ----------------------------------------------------------------
# 11. Use scikit-learn's LogisticRegression to train a simple model
# predicting readmission. Report the accuracy score on the test set.
# ----------------------------------------------------------------
model = LogisticRegression()
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)
print("\n\nAccuracy:", accuracy)
```

Output:

```
Accuracy: 1.0
```

Explanations:

This code fits a logistic regression model on the scaled training data and predicts readmission on the test set. Accuracy is then calculated to evaluate the model's performance. In this case, accuracy is 1.0, which means that the model predicted all readmission values correctly in the test set.

Because the dataset is very small and some missing values were removed, the model shows a perfect accuracy, but in larger datasets it could indicate overfitting.